



| **SCCC**

2026

송실대학교 프로그래밍 대회

SCON

문제지

Offical Problemset

언어 가이드

- 채점은 Vultr Optimized Cloud Compute 인스턴스(voc-c-8c-16gb-150s)에서 진행됩니다.
- 채점 서버의 운영체제는 Ubuntu 24.04 LTS x64 입니다.
- 아래 언어 중 원하는 언어를 선택해 사용할 수 있습니다.
 - C11: gcc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
 - C++17: g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
 - Java 21: openjdk version "21.0.10" 2026-01-20
 - Python 3: Python 3.9.18
 - PyPy3: PyPy 7.3.15 with GCC 13.2.0
- C11/C++17에서 `scanf_s`와 `Windows.h`등의 비표준 함수를 사용할 수 없습니다.
- Java를 사용하는 경우, `main` 메소드를 포함하는 클래스의 이름은 `Main`이어야 합니다.
- Python에서 `numpy`와 같은 외부 모듈을 사용할 수 없습니다.
- 채점 사이트에서 컴파일 에러를 받은 경우, '컴파일 에러' 글씨를 누르면 오류가 발생한 위치를 볼 수 있습니다.
- 아래 코드는 표준 입력(standard input)을 통해 공백으로 구분된 두 정수를 입력으로 받아서 표준 출력(standard output)을 통해 합을 출력하는 코드입니다.

– C11

```
1  #include <stdio.h>
2
3  int main() {
4      int a, b;
5      scanf("%d %d", &a, &b);
6      printf("%d\n", a + b);
7      return 0;
8  }
```

– C++17

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int a, b;
6      cin >> a >> b;
7      cout << a + b << endl;
8      return 0;
9  }
```

– Java 15

```
1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          int a = sc.nextInt();
```

```
7         int b = sc.nextInt();
8         System.out.println(a + b);
9         sc.close();
10    }
11 }
```

– Python 3 / PyPy3

```
1 a, b = map(int, input().split())
2 print(a + b)
```

- 입출력 양이 많을 때는 위 코드를 사용한 입출력이 너무 오래 걸리기 때문에 다른 방식으로 입출력해야 합니다.
- C11/C++17에서 `scanf`와 `printf`를 사용하는 경우, 입출력 속도는 문제를 해결할 수 있을 정도로 충분히 빠릅니다.
- C++17에서 `cin`과 `cout`을 사용하는 경우, 입출력 전에 `ios_base::sync_with_stdio(false);`와 `cin.tie(nullptr);`를 사용하여야 합니다. 단, 이 이후에는 `cin`, `cout` 계열 함수와 `scanf`, `printf` 계열 함수를 섞어서 사용하면 안 됩니다. 또한, 개행문자로 `std::endl` 대신 `"\n"`을 사용해 주세요.
- Java 15에서는 `BufferedReader`와 `BufferedWriter`를 사용하여야 합니다.
- Python 3 및 PyPy3에서는 `input()` 대신 `sys.stdin.readline().rstrip("\n")`을 사용하여야 합니다. 코드의 가장 위 부분에 `import sys`와 `input = lambda: sys.stdin.readline().rstrip("\n")`을 사용하여야 합니다.
- 아래 코드는 표준 입력(standard input)을 통해 문제의 개수 T 를 입력받은 다음 T 줄에 걸쳐 공백으로 구분된 두 정수를 입력으로 받아 표준 출력(standard output)을 통해 두 정수의 합을 총 T 줄에 걸쳐 출력하는 코드입니다.

– C++17

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7     int T;
8     cin >> T;
9     for(int i=1; i<=T; i++){
10         int a, b;
11         cin >> a >> b;
12         cout << a + b << "\n"; // do not use endl
13     }
14     return 0;
15 }
```

– Java 15

```
1 import java.util.*;
2 import java.io.*;
3
4 public class Main{
```

```
5     public static void main(String[] args) throws IOException {
6         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
7         BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
8
9         int T = Integer.parseInt(br.readLine());
10        for(int i=1; i<=T; i++){
11            String[] temp = br.readLine().split(" ");
12            int a = Integer.parseInt(temp[0]);
13            int b = Integer.parseInt(temp[1]);
14            bw.write(String.valueOf(a + b) + "\n");
15        }
16        br.close();
17        bw.close();
18    }
19 }
```

– Python 3 / PyPy3

```
1 import sys
2 input = lambda: sys.stdin.readline().rstrip("\n")
3
4 T = int(input())
5 for _ in range(T):
6     a, b = map(int, input().split())
7     print(a + b)
```

대회 중 유의 사항

- 2026 숭실대학교 프로그래밍 대회(2026 SCON) 유의 사항입니다.
- 대회 정보
 - 이 대회는 숭실대학교 IT대학이 주최, 컴퓨터학부 문제해결 소모임 SCCC가 주관합니다.
- 사진 촬영 안내
 - 대회 당일 현장 스태프가 대회 현장을 촬영하여 사진으로 기록 및 온라인 게시할 예정입니다.
 - 사진은 추후 숭실대학교, 숭실대학교 IT대학, IT대학 소속 학부, SCCC의 홍보와 SCCC가 주관하는 대회 홍보에 사용될 수 있습니다.
- 대회 진행 관련
 - 대회는 4시간 동안 12문제로 진행됩니다.
 - 7문제 이상 해결한 팀은 스코어보드 갱신이 중단됩니다.
 - 대회 종료 1시간 전부터 모든 팀의 스코어보드 갱신이 중단됩니다.
 - 대회 도중에는 대회 스태프와 팀원을 제외한 타인과 대화할 수 없습니다.
 - 대회 사이트 및 언어 공식 레퍼런스 사이트를 제외한 모든 인터넷의 사용은 금지됩니다.
 - * C/C++ : <https://en.cppreference.com/w/>
 - * Java : <https://docs.oracle.com/en/java/javase/15/docs/api/>
 - * Python : <https://docs.python.org/3/>
 - 화장실 이용 시 스태프와 동행해야 하며, 한 번에 한 명씩 이용 가능합니다.
 - 대회 도중 휴대전화 사용이 불가능합니다. 불가피한 경우 스태프의 감독하에 사용 가능합니다.
 - 문제와 관련된 질문은 대회 페이지의 '질문' 탭을 이용해야 합니다. 현장 스태프는 문제에 대한 질문을 받지 않습니다.
 - 대회 공지는 대회 페이지의 '공지' 탭을 이용해 전달합니다. 주기적으로 확인해 주시길 바랍니다.
- 문제 관련
 - 문제는 운영진들이 생각하는 난이도순으로 정렬되어 있지만, 모든 문제를 읽고 고민하는 것을 권장합니다.
 - 모든 문제는 C++, Java, PyPy3으로 해결할 수 있음이 보장됩니다. (Python3는 보장하지 않음)
 - 모든 문제의 메모리 제한은 1024MB로 동일합니다.
 - 제출한 프로그램은 문제에 명시된 제한 시간 내에 정답을 출력하고 정상적으로 종료되어야 합니다. 이는 return code가 0이어야 함을 의미하여, 이외의 exit code는 런타임 에러가 발생합니다.
 - 제출한 프로그램은 문제에 명시된 제한 메모리보다 많은 메모리를 사용할 수 없습니다.
 - 언어별 추가 시간과 추가 메모리가 주어지지 않습니다.
 - 제출한 프로그램은 표준 입력(standard input)을 통해 입력받아서 표준 출력(standard output)을 통해 정답을 출력해야 합니다.
 - 표준 입출력을 제외한 파일 입출력, 네트워킹, 멀티 스레딩 등의 시스템 콜은 사용할 수 없습니다.

- 팀의 등수는 다음과 같은 방법을 이용해 계산합니다.
 - 문제의 패널티 = (대회가 시작한 시점으로부터 처음으로 **맞았습니다!!**를 받기까지 걸린 분 단위 시간) + (제출 횟수 - 1) * 20분
 - 팀의 패널티 = **맞았습니다!!**를 받은 모든 문제의 패널티의 합
 - 팀의 등수 = (더 많은 문제를 푼 팀의 수) + (푼 문제의 개수가 동일하면서 패널티가 더 작은 팀의 수) + 1
 - 컴파일 에러는 패널티에 포함되지 않습니다.
- 문제 관련 질문에 대한 답변은 다음 중 하나로 주어집니다.
 - 문제를 잘 읽어주시길 바랍니다 / **Read the problem statement**
질문에 대한 답변이 문제 지문에 있다는 의미입니다.
 - 답변할 수 없습니다 / **No comments**
질문이 잘못되었거나 답변으로 인해 대회 공정성이 해쳐지는 등 답변하기 힘들다는 의미입니다.
 - 전체 공지사항을 참고하시길 바랍니다 / **Please refer to the announcement**
답변 대신 전체 공지사항으로 공지한다는 의미입니다.
 - 네 / **Yes**
 - 아니요 / **No**
 - 기타 답변

간식 안내

- 제공되는 간식에 알레르기 유발 물질이 포함되어 있으니 주의해서 섭취하시길 바랍니다.
- 영양성분표 및 원재료명 관련 문의는 대회 페이지의 질문 기능을 통해 질문하면 답변드리겠습니다.
- 오예스
 - 우유, 밀, 계란, 대두 함유
 - 땅콩을 사용한 제품과 같은 시설에서 제조
- 쿠크다스 케이크
 - 밀, 우유, 대두, 계란, 돼지고기 함유
- 트웝스 미니
 - 밀, 우유, 대두 함유
 - 땅콩을 사용한 제품과 같은 시설에서 제조
- 비쵸비
 - 밀, 우유, 대두, 쇠고기 함유
 - 달걀, 땅콩, 호두, 복숭아, 토마토, 돼지고기, 닭고기, 오징어, 새우, 게, 조개류(굴, 홍합) 혼입 가능
- 허쉬 쿼엔크 미니
 - 밀, 우유, 대두 함유
- 오리온 카스타드
 - 밀, 계란, 대두, 우유 함유
 - 땅콩, 호두, 복숭아, 토마토, 돼지고기, 쇠고기, 닭고기, 오징어, 새우, 게, 조개류(굴, 홍합) 혼입 가능

2026 Soongsil Programming Contest

Problem List

#	Problem Name	Time limit	Memory limit	Page
A	Where is my “ICP”	1 second	1024MiB	8 – 9
B	송실대입구역	1 seconds	1024MiB	10 – 10
C	크림 먼저, 잼 먼저	1 second	1024MiB	11 – 12
D	사전순 최소	1 second	1024MiB	13 – 14
E	구간 뒤집기	1 second	1024MiB	15 – 15
F	덜 개성있는 구간	1 second	1024MiB	16 – 16
G	보안	1 second	1024MiB	17 – 18
H	갈게 만들기	1 second	1024MiB	19 – 20
I	분탕의 신	1 second	1024MiB	21 – 23
J	SCON 마법진	1 seconds	1024MiB	24 – 26
K	XOR 펠린드롬	2 seconds	1024MiB	27 – 28
L	뱀 수열	3 seconds	1024MiB	29 – 30

문제지에 있는 문제가 총 12문제가 맞는지 확인하시길 바랍니다.

문제는 출제진이 생각하는 난이도순으로 정렬되어 있지만, 모든 문제를 읽고 고민하는 것을 권장합니다.

모든 문제는 C++17, Java 15, PyPy3으로 풀 수 있음을 보장합니다. (단, Python 3는 보장하지 않음)

Problem A. Where is my “ICP”

Time limit: 1 초

Memory limit: 1024 megabytes



민성이는 2025년 ICPC 대만 리저널에 참여하고 받은 에코백을 보며 슬퍼하고 있다. 원래 에코백에는 ICPC 네 글자가 멋지게 인쇄되어 있었지만, 몇 글자가 떨어져 나가고 문자열 S 만 남았기 때문이다.

슬퍼하는 민성이를 위해, 창민이는 지난 SCOP 운영진이 남겨 둔 문자 스티커 T 의 일부를 사용해 잃어버린 글자를 채워 가방에 다시 ICPC 문자열을 만들어 주려 한다.

문자열 S 와 T 가 주어졌을 때, 창민이가 민성이의 가방에 ICPC 문자열을 복원할 수 있는지 판별하라.

Input

첫째 줄에 가방에 남아 있는 문자열 S 의 길이를 나타내는 정수 N 이 주어진다.

둘째 줄에 길이가 N 인 문자열 S 가 주어진다.

셋째 줄에 문자 스티커 T 의 길이를 나타내는 정수 M 이 주어진다.

넷째 줄에 길이가 M 인 문자열 T 가 주어진다.

Output

T 에 포함된 스티커를 사용하여 ICPC를 복원할 수 있으면 `m4us happy`를, 그렇지 않으면 `m4us sad`를 출력한다.

Constraints

- $1 \leq N \leq 4$
- $1 \leq M \leq 100$
- S 는 I, C, P로만 구성되어 있다.
- S 는 ICPC의 부분 수열(subsequence)이다. 즉, S 는 ICPC에서 순서를 유지한 채 일부 문자가 사라진 문자열이다.
- T 는 알파벳 대문자로만 구성되어 있다.
- T 에 같은 알파벳이 여러 번 등장할 수 있으며, 등장한 횟수 만큼 해당 알파벳의 스티커를 사용할 수 있다.

Example

표준 입력(stdin)	표준 출력(stdout)	Notes
3 CPC 3 ABC	m4us sad	<i>S</i> 문자열인 CPC에서 ICPC를 만들려면 I가 하나 필요하지만, <i>T</i> 문자열인 ABC에는 I가 없으므로 m4us sad를 출력한다.
2 IC 11 CPISTOOHARD	m4us happy	<i>S</i> 문자열인 IC에서 ICPC를 만들려면 P와 C가 하나씩 더 필요한 상태이다. <i>T</i> 문자열인 CPISTOOHARD에는 P와 C가 각각 1개 이상 존재하므로 m4us happy를 출력한다.

Hints

C/C++ 에서 문자열을 저장하기 위해 `char` 배열을 사용하는 경우, 문자열의 끝을 나타내는 널 문자(`\0`)를 저장할 공간이 추가로 필요하다. 예를 들어 길이가 100인 문자열을 저장하려면 배열의 크기를 101 이상으로 선언해야 한다.

Notes

실제로 창민이는 지문과 다르게 민성이를 놀렸고, 그 때문인지 창민이는 2026 ICPC Asia Pacific Championship에 참가하고 받은 가방에 붙어있던 스티커가 떨어졌다.

Problem B. 승실대입구역

Time limit: 1 초
Memory limit: 1024 megabytes

지하철을 타고 등교하는 도현이는 매년 승실대입구역에서 내린다. 승실대입구역에서 출구로 나가기 위해서는 3개의 에스컬레이터를 타야 한다.

길이가 L 인 에스컬레이터는 가만히 서서 올라가면 L 초가 걸리고, 체력을 소모하지 않는다. 길이가 L 인 에스컬레이터를 걸어서 올라가면 $\frac{L}{2}$ 초 만에 올라갈 수 있지만, $\frac{L}{2}$ 만큼의 체력을 소모한다. 단, 에스컬레이터를 타는 도중에 올라가는 방법을 바꿀 수는 없다.

도현이는 현재 H 의 체력을 가지고 있다. 체력을 전부 소모하면 학교에 가서 공부를 할 수 없으므로 남은 체력이 1 이상이 되도록 올라가려고 한다.

3개의 에스컬레이터의 길이 A, B, C 와 체력 H 가 주어졌을 때, 도현이가 체력을 1 이상 남기면서 가장 빠르게 출구로 나갔을 때 걸리는 시간을 출력하는 프로그램을 작성하라.

에스컬레이터의 길이는 항상 짝수로 주어지며, 정답은 정수임이 보장된다.

Input

첫째 줄에 에스컬레이터의 길이 A, B, C 가 공백으로 구분되어 주어진다.

둘째 줄에 도현이의 체력 H 가 주어진다.

Output

도현이가 체력을 1 이상 남기면서 가장 빠르게 출구로 나갔을 때 걸리는 시간을 출력한다.

Constraints

- 주어지는 모든 수는 정수이다.
- $2 \leq A, B, C \leq 200$
- $1 \leq H \leq 200$
- A, B, C 는 짝수이다.

Example

표준 입력(stdin)	표준 출력(stdout)
200 120 60 102	280
70 60 88 30	218

Notes

첫 번째 예제에서 길이가 200인 에스컬레이터를 걸어서 올라가고, 나머지 에스컬레이터는 가만히 서서 올라가면, 도현이의 체력을 2 남긴 채로 280초 만에 출구로 나갈 수 있다.

Problem C. 크림 먼저, 잼 먼저

Time limit: 1 초
Memory limit: 1024 megabytes

영국에서 스콘을 맛있게 먹는 방법은 위에 크림을 바른 후 잼을 얹는 것, 혹은 위에 잼을 바른 후 크림을 얹는 것이다. 성준이는 파티에서 어느 방식의 스콘이 더 많은지 세어 보기로 했다. 불행하게도, 모든 스콘들은 하나의 탑으로 쌓여 있어, 위에서부터 재료를 읽을 수밖에 없었다.

스콘 탑의 재료들을 나타내는 문자열 T 가 주어지면, 크림이 위인 스콘의 개수와, 잼이 위인 스콘의 개수를 각각 구하여라.

문자열 T 는 문자 C, J, S 로만 이루어져 있다. C 는 크림을, J 는 잼을, S 는 스콘을 나타낸다.

- 크림이 위인 스콘: 1개 이상의 C , 1개 이상의 J , 하나의 S 가 차례대로 연속해서 나타나는 경우이다.
- 잼이 위인 스콘: 1개 이상의 J , 1개 이상의 C , 하나의 S 가 차례대로 연속해서 나타나는 경우이다.
- C 와 J 는 모두 자신의 오른쪽에서 가장 처음 오는 S 에 바르는 것으로 가정한다. 즉, $CJCS$ 에서 S 에는 C , J , C 가 모두 발라져 있다.

만약 크림이나 잼 중 하나라도 빠져 있거나, 두 재료가 번갈아 여러 번 나타나는 스콘은 잘못된 스콘으로 간주하며, 어느 쪽에도 세지 않는다.

Input

첫째 줄에 문자열 T 의 길이를 나타내는 정수 N 이 주어진다.

둘째 줄에 길이가 N 인 문자열 T 가 주어진다.

Output

첫째 줄에 크림이 위인 스콘의 개수를 출력한다.

둘째 줄에 잼이 위인 스콘의 개수를 출력한다.

Constraints

- $1 \leq N \leq 200$
- 문자열 T 는 C, J, S 로만 구성되어 있으며, 반드시 S 로 끝난다.

Example

표준 입력(stdin)	표준 출력(stdout)
3 CJS	1 0
5 CCJJS	1 0
13 JCSCJCSJJCCS	0 2
28 SJJJCSCJSJSCJCSJJCCJJJCJSSJS	1 1

Notes

예제 4의 문자열 $T = \text{SJJJCSCJSJSCJCSJJCCJJJCJSSJS}$ 를 S 를 기준으로 나누면

$S, JJCS, CJS, JS, CJCS, JJCCJJCS, S, JS$

를 얻는다.

이 중 CJS 는 크림이 위인 스콘 1개, $JJCS$ 는 잼이 위인 스콘 1개이다. 나머지 $S, JS, CJCS, JJCCJJCS, S, JS$ 는 재료가 빠져 있거나 재료가 여러 번 번갈아 나타나므로 모두 잘못된 스콘이다.

따라서 출력은 첫째 줄에 1, 둘째 줄에 1이다.

Problem D. 사전순 최소

Time limit: 1 초

Memory limit: 1024 megabytes

길이 N 인 문자열 T 가 주어진다. T 는 알파벳 소문자로만 이루어져 있다.

당신은 다음 연산을 0회 이상 사용할 수 있다.

- 두 정수 l, r 를 선택한다. ($1 \leq l < r \leq |T|$)
- 만약 부분 문자열 $T_l T_{l+1} \dots T_r$ 의 모든 문자가 동일하다면, 그 구간에서 문자 하나만 남기고 나머지를 모두 제거한다.

즉, $T_l = T_{l+1} = \dots = T_r = c$ 인 경우,

문자열 $T_1 T_2 \dots T_{l-1} \underline{T_l T_{l+1} \dots T_r} T_{r+1} \dots T_{|T|}$ 를 $T_1 T_2 \dots T_{l-1} c T_{r+1} \dots T_{|T|}$ 로 바꿀 수 있다.

이 연산을 0회 이상 원하는 만큼 사용하여 만들 수 있는 모든 문자열 중, 사전 순으로 가장 앞서는 문자열을 구하여라.

Input

첫째 줄에 문자열 T 의 길이를 나타내는 정수 N 이 주어진다.

둘째 줄에 길이가 N 인 문자열 T 가 주어진다.

Output

가능한 모든 결과 문자열 중 사전 순으로 가장 앞서는 문자열을 출력한다.

Constraints

- $1 \leq N \leq 200\,000$
- T 는 알파벳 소문자로만 이루어져 있다.

Example

표준 입력(stdin)	표준 출력(stdout)
6 aabbba	aaba
5 bbacc	bac

Notes

사전 순 비교는 다음과 같이 정의한다.

두 문자열 A, B 에 대하여, 다음 둘 중 하나를 만족하면 A 가 B 보다 사전 순으로 앞선다고 한다.

- A 가 B 의 접두사인 경우
- 처음으로 다른 위치를 i 라고 할 때, A_i 가 B_i 보다 알파벳 순으로 앞서는 경우

예를 들어, ab 는 aba 보다 사전 순으로 앞서고, abc 는 abd 보다 사전 순으로 앞선다.

어떤 구간에 연산을 적용할 수 있더라도 반드시 적용해야 하는 것은 아니다.

예제 1의 문자열 $aabbba$ 의 경우, 만들 수 있는 문자열은 $aabbba, abbba, aabba, abba, aaba, aba$ 의 6가지이다.

이 중 사전 순으로 가장 앞서는 문자열은 $aaba$ 이다.

Problem E. 구간 뒤집기

Time limit: 1 초

Memory limit: 1024 megabytes

길이 N 인 이진 문자열 T 가 주어진다. 다음 연산을 원하는 만큼 사용할 수 있다.

- 두 정수 l, r ($1 \leq l \leq r \leq N$)를 선택한다.
- 부분 문자열 $T_l T_{l+1} \cdots T_r$ 의 모든 문자를 뒤집는다. 즉, 0은 1로, 1은 0으로 바뀐다.

문자열 T 를 다음 조건을 만족하도록 바꾸고자 한다.

- 모든 $1 \leq i < N$ 에 대해 $T_i \neq T_{i+1}$

즉, 인접한 두 문자가 항상 서로 달라야 한다.

조건을 만족시키기 위해 필요한 연산의 최소 횟수를 구하여라.

Input

첫째 줄에 이진 문자열 T 의 길이를 나타내는 정수 N 이 주어진다.

둘째 줄에 길이가 N 인 이진 문자열 T 가 주어진다.

Output

조건을 만족시키기 위해 필요한 연산의 최소 횟수를 출력한다.

Constraints

- $1 \leq N \leq 200\,000$
- T 는 0과 1로만 이루어져 있다.

Example

표준 입력(stdin)	표준 출력(stdout)
3 101	0
4 0011	1

Problem F. 덜 개성있는 구간

Time limit: 1 초
Memory limit: 1024 megabytes

다음 조건을 만족하는 길이가 N 인 수열 $A = [A_1, A_2, \dots, A_N]$ 를 아무거나 하나 찾아라.

- 수열 A 의 모든 원소는 서로 다르다.
- $1 \leq L \leq R \leq N$ 을 만족하는 모든 정수 쌍 (L, R) 에 대하여, $\sum_{i=L}^R A_i$ 가 가질 수 있는 서로 다른 값의 개수는 정확히 N 개이다.

주어진 범위 내에서 조건을 만족하는 수열은 항상 존재한다.

Input

첫째 줄에 수열 A 의 길이를 나타내는 정수 N 이 주어진다.

Output

첫째 줄에 조건을 만족하는 수열 A 의 원소들을 공백으로 구분하여 출력한다.

위 조건을 만족하는 수열이 여러 개라면 그중 아무거나 출력한다.

Constraints

- 주어지는 모든 수는 정수이다.
- $1 \leq N \leq 5000$
- $1 \leq i \leq N$ 인 모든 i 에 대해 $-10^9 \leq A_i \leq 10^9$ 이다.

Example

표준 입력(stdin)	표준 출력(stdout)
2	0 1
3	-2 0 2

Notes

수열 $A = [-2, 0, 2]$ 에서 모든 연속 구간의 합을 구하면 다음과 같다.

- 2: (1, 1), (1, 2)
- 0: (1, 3), (2, 2)
- 2: (2, 3), (3, 3)

Problem G. 보안

Time limit: 1 초
Memory limit: 1024 megabytes

SCCC 회장 문성이는 이번 SCON 대회의 서버 운영을 맡고 있다.

대회가 시작되면 서버에는 시간순으로 N 개의 요청이 들어오고, 이를 앞에서부터 순서대로 처리해야 한다. i 번째 요청에는 안정도 A_i 가 주어진다.

문성이는 이번 대회에서 사용할 하나의 정수 신뢰도 T 를 정하려고 한다. 기준값이 정해지면 각 요청은 다음과 같이 분류된다.

- $A_i \geq T$ 이면 해당 요청은 정상 요청이다.
- $A_i < T$ 이면 해당 요청은 위험 요청이다.

서버에는 운영 여유도 `credit` 이 있으며, 초기값은 0 이다. 요청을 하나 처리할 때마다 `credit` 은 다음과 같이 변한다.

- 요청이 정상 요청이면 `credit` 이 1 증가한다.
- 요청이 위험 요청이면 `credit` 이 1 감소한다.

처리 도중 어느 순간이라도 `credit` 이 음수가 되면 서버는 버티지 못하고 즉시 다운된다.

문성이는 서버를 안정적으로 운영하기 위해 B 만큼의 예산을 사용할 수 있다. 위험 요청을 처리하는 순간에 긴급 패치를 적용하면, 해당 요청을 정상 요청으로 바꿀 수 있다. 만약 i 번째 요청이 위험 요청이었다면, 이 요청을 패치하는 데 필요한 비용은 $T - A_i$ 이다.

패치 비용의 총합은 B 를 초과할 수 없다.

문성이가 SCON 서버를 끝까지 한 번도 다운시키지 않도록 할 수 있는 최대의 정수 신뢰도 T 를 구하여라.

Input

첫째 줄에 요청의 개수 N 과 사용할 수 있는 총 예산 B 가 공백으로 구분되어 주어진다.

둘째 줄에 N 개의 정수 $A_1, A_2, \dots, A_N$ 이 공백으로 구분되어 주어진다.

Output

조건을 만족하도록 선택할 수 있는 최대의 정수 신뢰도 T 를 출력한다.

Constraints

- 주어지는 모든 수는 정수이다.
- $1 \leq N \leq 200\,000$
- $0 \leq B \leq 10^9$
- $1 \leq i \leq N$ 인 모든 i 에 대해 $1 \leq A_i \leq 10^9$ 이다.

Example

표준 입력(stdin)	표준 출력(stdout)
6 4 2 6 2 1 4 4	5

Notes

신뢰도가 5일때 1, 5번째 요청을 $(5 - 2) + (5 - 4) = 4$ 의 비용으로 긴급 패치하면 `credit`의 값은 순서대로 1, 2, 1, 0, 1, 0이 되어 서버를 끝까지 다운시키지 않을 수 있고, 설정 할 수 있는 최대 신뢰도이다.

Problem H. 같게 만들기

Time limit: 1 초

Memory limit: 1024 megabytes

길이가 N 인 정수 수열 A 와 Q 개의 쿼리가 주어진다. 각 쿼리는 다음과 같은 형태이다.

- $i\ x$: A_i 를 x 로 변경한다.

초기 수열에 대해, 그리고 각 쿼리를 처리한 직후 수열에 대해 각 수열을 같게 만들 수 있는지 판별하라.

어떤 수열 B 를 같게 만들 수 있다는 것은, 다음 연산을 0회 이상 사용해서 수열의 모든 원소를 같은 값으로 만들 수 있다는 의미이다.

- 두 정수 i, j 를 고른다. i 와 j 가 같을 수 있다. ($1 \leq i, j < N$)
- 그 후 수열을 다음과 같이 변경한다.
 1. B_i 를 1 증가시킨다.
 2. B_{i+1} 을 1 감소시킨다.
 3. B_j 를 1 감소시킨다.
 4. B_{j+1} 을 1 증가시킨다.

Input

첫째 줄에 수열 A 의 길이를 나타내는 정수 N 이 주어진다.

둘째 줄에 N 개의 정수 $A_1, A_2, \dots, A_N$ 이 공백으로 구분되어 주어진다.

셋째 줄에 정수 Q 가 주어진다.

이후 Q 개의 줄에 걸쳐 두 정수 i, x 가 공백으로 구분되어 주어진다. 이는 A_i 를 x 로 변경하는 것을 의미한다.

Output

총 $Q + 1$ 개의 줄을 출력한다.

첫째 줄에는 초기 수열을 같게 만들 수 있으면 Yes를, 그렇지 않으면 No를 출력한다.

그 다음 Q 개의 줄에는 각 쿼리를 처리한 직후의 수열에 대해, 같게 만들 수 있으면 Yes를, 그렇지 않으면 No를 출력한다.

Constraints

- 주어지는 모든 수는 정수이다.
- $2 \leq N \leq 200\,000$
- $1 \leq Q \leq 200\,000$
- $1 \leq j \leq N$ 인 모든 j 에 대해 $1 \leq A_j \leq 10^8$
- 모든 쿼리에 대해 $1 \leq i \leq N$, $1 \leq x \leq 10^8$

Example

표준 입력(stdin)	표준 출력(stdout)
4 3 1 1 3 2 2 3 3 3	Yes No Yes
5 5 4 3 2 1 1 5 6	No Yes

Notes

초기 수열이 $A = [3, 1, 1, 3]$ 일 때, $i = 3, j = 1$ 을 선택하여 연산을 한 번 수행하면 수열은 $A' = [2, 2, 2, 2]$ 가 된다. 따라서 초기 수열을 같게 만들 수 있다.

1번 쿼리로 $A_2 = 3$ 을 적용하면 수열은 $A = [3, 3, 1, 3]$ 으로 변경된다. 이 경우에는 수열을 같게 만들 수 없다.

2번 쿼리로 $A_3 = 3$ 을 적용하면 수열은 $A = [3, 3, 3, 3]$ 으로 변경된다. 이 경우에는 수열을 같게 만들 수 있다.

Problem I. 분탕의 신

Time limit: 1 초

Memory limit: 1024 megabytes

이 문제는 인터랙티브 문제이다.

숭실대학교 알고리즘 동아리 SCCC에는 전통적으로 내려오는 길이 N 의 이진 수열 $A = [A_1, A_2, \dots, A_N]$ 족보 코드가 있다. 각 원소는 0 또는 1이다.

분탕의 신 문성이는 족보를 싫어해 SCCC 부원들이 족보 코드를 쉽게 알아내지 못하도록 수열을 계속 변경하는 분탕을 친다.

당신의 목표는 채점기와 상호작용하여 문성이의 분탕을 이겨내 현재 수열 A 를 정확히 알아내는 것이다.

당신은 채점기에게 최대 1333번까지 질문할 수 있다. 한 번의 질문은 다음 형식으로 한다.

? i

이 질문을 하면 채점기는 현재 수열의 i 번째 원소 A_i 의 값을 반환한다.

하지만 문성이는 호락호락하지 않다. 채점기는 **10번째, 20번째, 30번째, ...** 질문에 답한 직후마다 현재 수열에 대해 다음 네 가지 연산 중 하나를 적용한다.

- 아무 연산도 하지 않음
- 전체 수열을 flip
- 전체 수열을 reverse
- 전체 수열을 flip한 뒤 reverse

flip과 reverse의 정의는 아래 노트를 참고하라.

현재 수열을 정확히 알아냈다면, 다음 형식으로 답을 출력해야 한다.

! $A_1 A_2 \dots A_N$

이때 출력한 수열은 출력하는 순간의 현재 수열과 정확히 일치해야 한다.

Interaction Protocol

상호작용이 시작되면 첫째 줄에 정수 N 이 주어진다.

그 후, 참가자는 최대 1333번까지 질문할 수 있다.

질문은

$? \ i$

형식으로 출력해야 하며, 끝에 개행을 추가해야 한다. 여기서 i 는 $1 \leq i \leq N$ 을 만족하는 정수이다.

올바른 질문을 출력하면 채점기는 현재 수열의 i 번째 값인 A_i 를 반환한다. 반환되는 값은 항상 0 또는 1이다.

참가자는 최종적으로

$! \ A_1 \ A_2 \ \dots \ A_N$

형식으로 답을 출력해야 한다. 여기서 $A_1, A_2, \dots, A_N$ 은 반드시 공백으로 구분하여 출력해야 하며, 끝에 개행을 추가해야 한다. 즉, $! \ 0 \ 1 \ 0 \ 1$ 과 같은 형식으로 출력해야 하며, $! \ 0101$ 처럼 붙여서 출력해서는 안 된다.

출력한 수열은 출력하는 순간의 현재 수열과 정확히 일치해야 한다.

질문 형식이 잘못되었거나, i 가 범위를 벗어났거나, 질문 횟수 제한을 초과하는 등 참가자가 프로토콜을 어기면, 인터랙터는 -1을 출력한 뒤 즉시 종료한다. 참가자는 -1을 입력받는 즉시 프로그램을 종료해야 한다. 이 경우, **틀렸습니다**를 받는다.

최종 정답의 형식이 잘못되었거나, 출력한 수열이 현재 수열과 다르면 **틀렸습니다**를 받는다.

출력을 즉시 채점 프로그램에 전달하기 위해, 질문이나 정답을 출력한 뒤에는 반드시 개행을 출력하고 출력 버퍼를 비워야 한다. 이를 지키지 않아 상호작용이 정상적으로 진행되지 않으면 **런타임 에러**나 **시간 초과**와 같은 판정을 받을 수 있다.

언어별로 출력 버퍼를 비우는 방법의 예시는 다음과 같다.

- C: `fflush(stdout);`
- C++: `std::cout.flush();`
- Java: `System.out.flush();`
- Python: `stdout.flush()`

정답을 출력한 뒤 프로그램은 즉시 종료해야 한다.

이 문제의 인터랙터는 적응적이지 않다. 즉, 초기에 $A_1, A_2, \dots, A_N$ 의 값은 정해져 있으며, 당신이 보내는 쿼리와 관계없이 미리 정해진 순서대로 수열을 변화시킨다.

Constraints

- 주어지는 모든 수는 정수이다.
- $1 \leq N \leq 1000$
- $1 \leq i \leq N$ 인 모든 i 에 대해 $A_i \in \{0, 1\}$

Example

표준 입력(stdin)	표준 출력(stdout)
4	? 1
1	? 2
0	? 3
1	? 4
0	! 1 0 1 0

Notes

- **flip**은 수열의 모든 원소를 뒤집는 연산이다. 즉, 각 i 에 대해 현재 값 A_i 를 $1 - A_i$ 로 바꾼다.
- **reverse**는 수열의 순서를 뒤집는 연산이다. 즉, 연산 후의 수열 B 는 각 i 에 대해 $B_i = A_{N+1-i}$ 를 만족한다.
- **flip** 후 **reverse**는 먼저 모든 원소를 뒤집은 뒤, 그 결과 수열의 순서를 뒤집는 연산이다. 즉, 연산 전 수열이 A 이고 연산 후 수열이 B 라면 각 i 에 대해 $B_i = 1 - A_{N+1-i}$ 이다.
- 채점기의 변형은 오직 10번째, 20번째, 30번째, ... 질문에 답한 직후에만 일어난다. 최종 답안 출력 ! $A_1 A_2 \dots A_N$ 은 질문 횟수에 포함되지 않는다. 예를 들어 9번째 질문까지 한 뒤 곧바로 정답을 출력한다면, 그 사이에 수열이 추가로 변하는 일은 없다.
- 채점기가 어떤 연산을 적용했는지는 참가자에게 알려지지 않는다.
- 최종적으로 출력해야 하는 것은 초기 수열이 아니라, 정답을 출력하는 시점의 현재 수열이다.

Problem J. SCON 마법진

Time limit: 1 초

Memory limit: 1024 megabytes

올해도 어김없이 SCCC의 회장이 된 문성이는 스콘을 완벽하게 구워내기 위해 마법진을 연구하고 있다. 이 마법진은 1번부터 N 번까지 번호가 매겨진 N 개의 룬 문자와, 서로 다른 두 룬 문자를 연결하는 M 개의 마나 회로로 구성되어 있다.

처음 마법진을 그렸을 때 모든 룬 문자의 마나는 0이다. 문성이는 맛있는 스콘을 완성하기 위해 마법진에 다음 두 가지 쿼리를 처리하며 상태를 기록해야 한다.

- 1 i x : i 번 룬 문자에 마나를 주입하여, 해당 룬 문자의 마나 에너지를 x 만큼 증가시킨다.
- 2 d : 연결된 마나 회로의 수가 정확히 d 개인 모든 룬 문자들을 활성화한다. 이후 활성화된 각 룬 문자와 마나 회로로 직접 연결된 룬 문자들의 마나 에너지 총합을 계산한다.
 - 하나의 룬 문자가 여러 개의 활성화된 룬 문자와 연결되었다면, 연결된 횟수만큼 중복해서 합산한다.
 - 룬 문자의 활성화는 에너지를 계산하는 동안 일시적으로만 이루어지며, 계산이 끝난 직후 활성화되었던 룬 문자들은 다시 원래의 상태로 돌아간다.

Input

첫째 줄에 룬 문자의 개수 N , 마나 회로의 개수 M , 쿼리의 개수 Q 가 공백으로 구분되어 주어진다.

다음 M 개의 줄에는 마나 회로가 연결하는 두 룬 문자의 번호 u, v 가 공백으로 구분되어 주어진다.

다음 Q 개의 줄에는 쿼리가 1 i x 또는 2 d 의 형태로 주어진다.

Output

2번 쿼리가 주어질 때마다, 조건에 맞춰 계산된 마나 에너지의 총합을 한 줄에 하나씩 출력한다.

조건을 만족하는 룬 문자가 존재하지 않거나, 마나 회로로 연결된 룬 문자가 없어 합산할 마나가 없다면 0을 출력한다.

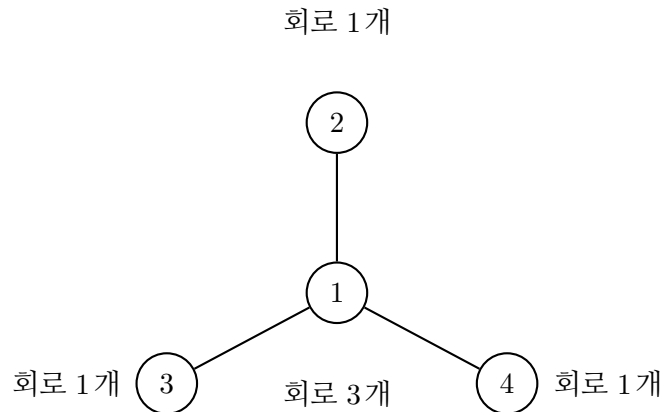
Constraints

- 주어지는 모든 수는 정수이다.
- $1 \leq N, Q \leq 100\,000$
- $0 \leq M \leq \min(\frac{N(N-1)}{2}, 200\,000)$
- $1 \leq u, v \leq N$
- $u \neq v$
- 같은 룬 문자 쌍을 연결하는 마나 회로가 여러 개 주어지지 않는다.
- 모든 1번 쿼리에 대해 $1 \leq i \leq N, 1 \leq x \leq 10^8$
- 모든 2번 쿼리에 대해 $0 \leq d \leq N - 1$
- 2번 쿼리는 반드시 1번 이상 주어진다.

Example

표준 입력(stdin)	표준 출력(stdout)
4 3 5 1 2 1 3 1 4 1 2 10 1 3 20 2 3 1 1 50 2 1	30 150
5 5 6 1 2 2 3 3 4 4 1 1 3 1 1 100 1 2 200 1 3 300 2 0 2 2 2 4	0 800 0

Notes



첫 번째 예제의 마법진에서 1번 룬에 연결된 마나 회로는 3개이며, 2, 3, 4번 룬에 연결된 마나 회로는 각각 1개이다. 각 쿼리에 따른 마나 에너지의 변화와 계산 과정은 다음과 같다.

- 1 2 10: 2번 룬의 마나를 10만큼 증가시킨다. (현재 마나 상태: $X_1 = 0, X_2 = 10, X_3 = 0, X_4 = 0$)
- 1 3 20: 3번 룬의 마나를 20만큼 증가시킨다. (현재 마나 상태: $X_1 = 0, X_2 = 10, X_3 = 20, X_4 = 0$)
- 2 3: 연결된 마나 회로가 정확히 3개인 룬은 **1번**이다. 1번 룬과 직접 연결된 룬(2, 3, 4번)의 마나 총합을 계산한다.
 - 계산: $X_2 + X_3 + X_4 = 10 + 20 + 0 = 30$
 - 결과인 30을 출력한다.
- 1 1 50: 1번 룬의 마나를 50만큼 증가시킨다. (현재 마나 상태: $X_1 = 50, X_2 = 10, X_3 = 20, X_4 = 0$)
- 2 1: 연결된 마나 회로가 정확히 1개인 룬은 **2, 3, 4번**이다. 활성화된 각 룬과 직접 연결된 룬의 마나를 모두 합산한다.
 - 2번 룬 활성화: 연결된 1번 룬의 마나($X_1 = 50$) 합산
 - 3번 룬 활성화: 연결된 1번 룬의 마나($X_1 = 50$) 합산
 - 4번 룬 활성화: 연결된 1번 룬의 마나($X_1 = 50$) 합산
 - 계산: $50 + 50 + 50 = 150$
 - 결과인 150을 출력한다.

Problem K. XOR 팰린드롬

Time limit: 2 초

Memory limit: 1024 megabytes

길이가 M 인 이진 문자열 N 개가 주어진다.

주어진 문자열 중 1개 이상을 선택하여, 선택한 모든 문자열을 비트별 배타적 논리합(bitwise XOR)한 결과 문자열이 팰린드롬이 되는 경우의 수를 구하여라.

단, 문자열을 선택하는 순서는 고려하지 않는다. 즉, 고르는 순서만 다르고 선택된 문자열의 구성이 같은 경우는 한 번만 센다. 또한, 입력으로 같은 이진 문자열이 여러 번 주어지더라도, 이들은 서로 구별되는 다른 문자열로 취급한다.

배타적 논리합(bitwise XOR) 또는 팰린드롬에 대한 설명은 하단의 노트를 참고하라.

Input

첫째 줄에 정수 N, M 이 공백으로 구분되어 주어진다.

둘째 줄부터 N 개의 줄에 걸쳐 길이가 M 인 이진 문자열 $T_1, T_2, \dots, T_N$ 이 한 줄에 하나씩 주어진다.

Output

조건을 만족하도록 문자열을 선택하는 경우의 수를 출력한다. 단, 답이 매우 커질 수 있으므로 $1\,000\,000\,007 (= 10^9 + 7)$ 로 나눈 나머지를 출력한다.

Constraints

- $1 \leq N \leq 50\,000$
- $1 \leq M \leq 20$
- $1 \leq i \leq N$ 인 모든 i 에 대해 $|T_i| = M$ 이다.
- $1 \leq i \leq N$ 인 모든 i 에 대해 T_i 는 0과 1로만 이루어져 있다.

Example

표준 입력(stdin)	표준 출력(stdout)
5 4 1100 0011 1010 0101 1110	7
6 5 11010 00000 10101 00101 10000 11111	31

Notes

두 이진 문자열 A, B 의 비트별 배타적 논리합(bitwise XOR) $A \oplus B$ 는 다음과 같이 정의된다.

- A 의 i 번째 글자와 B 의 i 번째 글자가 서로 다르면 $A \oplus B$ 의 i 번째 글자가 1이고, 같으면 0이다.

예를 들어 $1100 \oplus 1010$ 은 0110으로 계산한다.

k 개의 이진 문자열 $S_1, S_2, \dots, S_k$ 를 비트별 배타적 논리합한 결과는 $(\dots((S_1 \oplus S_2) \oplus S_3) \oplus \dots) \oplus S_k$ 로 정의된다. 이 연산의 결과는 $S_1, S_2, \dots, S_k$ 의 순서를 바꾸더라도 변하지 않음을 증명할 수 있다.

팰린드롬이란, 앞에서부터 읽으나 뒤에서부터 읽으나 같은 문자열을 말한다. 예를 들어, 010, 0110은 팰린드롬이며 011, 1011은 팰린드롬이 아니다.

Problem L. 뱀 수열

Time limit: 3 초
Memory limit: 1024 megabytes

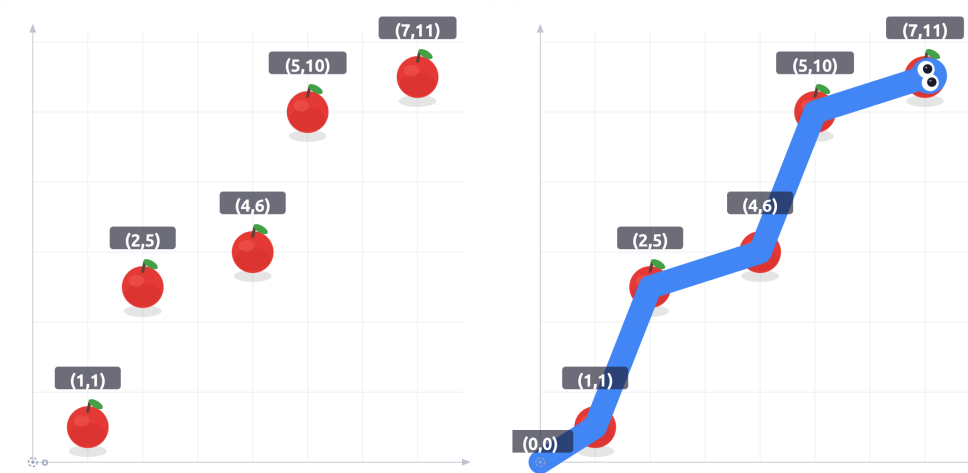
2차원 평면 위에 N 개의 사과가 놓여 있다.

한 마리의 뱀이 원점 $(0, 0)$ 에서 출발하여, 이 중 일부를 골라 순서대로 먹으며 이동한다. 뱀의 출발점을 $P_0 = (0, 0)$ 으로 놓고, 먹는 사과의 위치를 순서대로 $P_1, P_2, \dots, P_L$ 이라 하자. 각 점의 좌표를 $P_i = (x_i, y_i)$ 로 쓴다.

뱀은 오른쪽 위로만 전진한다. 즉, 모든 $0 \leq i < L$ 에 대해 $x_i < x_{i+1}$ 이고 $y_i < y_{i+1}$ 이다.

뱀은 지그재그로 나아간다. 연속한 두 점 사이의 이동 기울기를 $s_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$ ($0 \leq i < L$)라 정의하자. $L \geq 2$ 일 때, 기울기 수열 $s_0, s_1, \dots, s_{L-1}$ 은 다음 두 형태 중 하나를 만족해야 한다.

$$s_0 > s_1 < s_2 > s_3 < \dots \quad \text{또는} \quad s_0 < s_1 > s_2 < s_3 > \dots$$



예제 4의 사과 배치(왼쪽)와 최적 경로 $L = 5$ (오른쪽)를 나타낸 것이다.

뱀이 먹을 수 있는 사과의 최대 개수 L 을 구하여라.

Input

첫째 줄에 사과의 개수를 나타내는 정수 N 이 주어진다.

다음 N 개의 줄에 각각 사과의 x 좌표와 y 좌표를 의미하는 두 정수 x, y 가 공백으로 구분되어 주어진다.

Output

뱀이 먹을 수 있는 사과의 최대 개수를 출력한다.

Constraints

- 주어지는 모든 수는 정수이다.
- $1 \leq N \leq 3\,000$
- $1 \leq x, y \leq 10^9$
- 같은 좌표에 놓인 사과는 없다.

Example

표준 입력(stdin)	표준 출력(stdout)
3 1 3 2 2 3 1	1
2 1 2 3 3	2
5 1 1 3 2 6 4 10 8 15 15	3
5 1 1 2 5 4 6 5 10 7 11	5